

Quantum Error Correction - mini project

Edward Jones - 240224440

April 2025

1 Introduction

This report details a demonstration of an error correcting code using the 7-qubit Steane code and the stabilizer simulation method in QISKit [1,2]. An information qubit¹ was encoded into seven physical qubits. Stabilizer generator measurements were taken before and after a deliberate error. The two syndromes (measurement outcomes) were compared in order to diagnose the error and the appropriate Pauli gates applied for correction. A final measurement was taken to confirm the original syndrome was returned. The circuit presented is fault-tolerant only when the fault is an artificially inserted stochastic error that acts on a single qubit in a restricted range.

Any random error can occur during the operation of a quantum circuit. Quantum states, and therefore circuits, are very sensitive to noise (which causes error) and there are an infinite number of errors that can occur in a circuit [3]. The Steane code entangles ancillary qubits with the data qubit(s) which allows for measurement of the ancillary qubits, avoids direct measurement of the data qubit which would destroy the encoded information, and provides information about the type of error via the syndrome produced. The stabilizer measurements of the ancillary qubits decomposes the error into a linear combination of Pauli operators. Hence, the once infinite possible errors is now correctable with only four error types: no error, a bit-flip error, a phase-flip error, or a bit-and-phase flip error. This process of resolving a continuum of errors into four discrete possibilities is known as the discretization of errors.

The encoding of the data qubit was completed following the methods described in Quan et al.'s paper on the implementation of fault-tolerant encoding circuits using stabilizers and 'flag' bits in Steane code [4].

Finally, all code was written using QISKit 2.0.0.

2 Analysis of Circuit

2.1 Circuit Design

The circuit was set up using a combination of Quantum and Classical registers.

```
import numpy as np
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit_aer import AerSimulator

q = QuantumRegister(7, 'q') #Initialise 7 physical qubits.
stab_pre = QuantumRegister(6, 'stab_pre') # The 6 qubits forming the stabilizer generators.
stab_post = QuantumRegister(6, 'stab_post')
syn_pre_bits = [ClassicalRegister(1, f'pre_{i}') for i in range(6)] # Classical bits to store syndrome
syn_post_bits = [ClassicalRegister(1, f'post_{i}') for i in range(6)] # Classical bits to store syndrome
e_ancilla = QuantumRegister(6, 'e_ancilla') # Ancilla qubits for determining error.
error = ClassicalRegister(6, 'error')
stab_check = QuantumRegister(6, 'stab_check') # Confirmation measurements.
syn_check = ClassicalRegister(6, 'syn_check')

qc = QuantumCircuit(q, stab_pre, stab_post, e_ancilla, *syn_pre_bits, *syn_post_bits, error, stab_check, syn_check)
```

Figure 1: The circuit components including quantum registers and single and six bit classical registers. The asterisk on the single bit registers ensures they are passed to the quantum circuit separately.

There were a large number of qubits, classical bits and gates used throughout the circuit. There was the possibility of reducing the number of components required for the circuit, for example, the qubits from the initial stabilizer measurement could have been reset and reused for the subsequent measurements. However, the reset gate in QISKit, which initializes qubits in the state $|0\rangle$, and other gates were not included in the circuit in order to ensure that the circuit exclusively used Clifford gates.

¹The information qubit, or data qubit, is the qubit that holds the information that is to be transferred through the quantum circuit.

2.2 Encoding

The information qubit was encoded into 7 physical qubits via a series of CNOT and Hadamard gates.

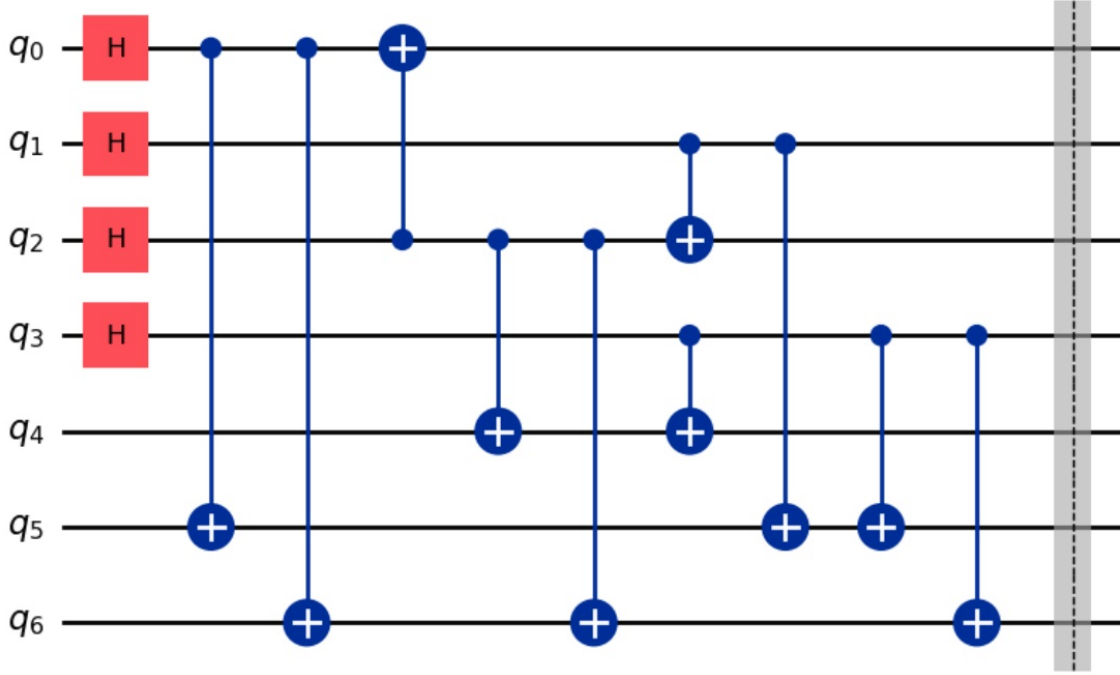


Figure 2: The encoding of the information qubit, q_0 . The other six qubits form the seven physical qubits. The barrier avoids automatic optimisation of the circuit ensuring the encoding is separate to the first stabilizer measurements.

The Hadamard gate acting on q_0 initializes the arbitrary information qubit in the $|+\rangle$ state. The encoding, originally introduced by Quan et al., is a solution to the typical large cost of error correction in Steane code. This encoding method avoids mid-circuit measurements and captures high-weight errors with flag qubits. The circuit presented here does not follow Quan et al.'s full process of fault-tolerant implementation and instead follows the traditional Steane code structure in the error correction stage - requiring a large number of qubits and quantum gates. Nonetheless, instead of simply initializing the gates in the $|0\rangle$ state, the encoding of an arbitrary superposition state offers a pedagogical demonstration of how logical qubits can be initialized in more complex and realistic states.

2.3 Stabilizers

The Steane code is defined, like many error-correcting codes, through its stabilizer group.

$$\begin{aligned} S_X^1 &= X_1 X_3 X_5 X_7, & S_X^2 &= X_2 X_3 X_6 X_7, & S_X^3 &= X_4 X_5 X_6 X_7. \\ S_Z^1 &= Z_1 Z_3 Z_5 Z_7, & S_Z^2 &= Z_2 Z_3 Z_6 Z_7, & S_Z^3 &= Z_4 Z_5 Z_6 Z_7. \end{aligned} \quad (1)$$

The stabilizer generators are measured before and after the error is manually introduced in order to diagnose the error through comparison of the two measurements. Stabilizers, by definition, stabilize the codespace meaning that the only states secured in this subspace are their $+1$ eigenstates.

$$S|\psi\rangle = +1|\psi\rangle. \quad (2)$$

However, the AerSimulator used for running the circuit simulates inherent noise in the circuit, resulting in the stabilizer measurements sporadically returning a -1 eigenvalue. This -1 eigenvalue represents a deviation from the codespace that is protected from error.

The S_Z and S_X stabilizer generators are measured to make the initial measurement.

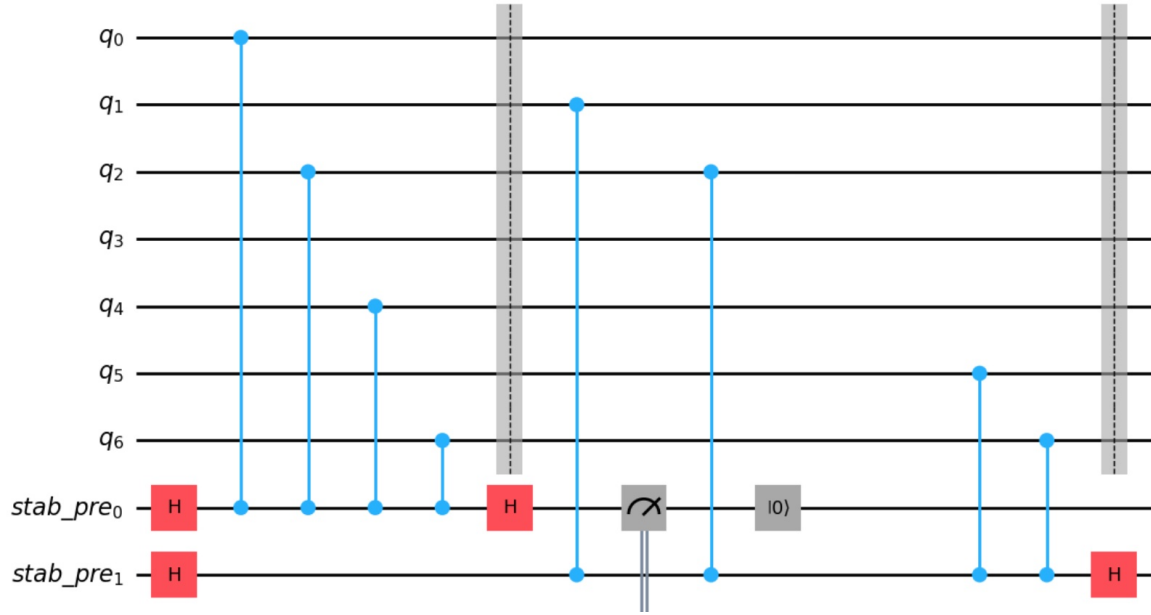


Figure 3: The S_Z^1 and S_Z^2 ($stab_pre_0$ and $stab_pre_1$ respectively) stabilizer generators applied to the 7 physical qubits.

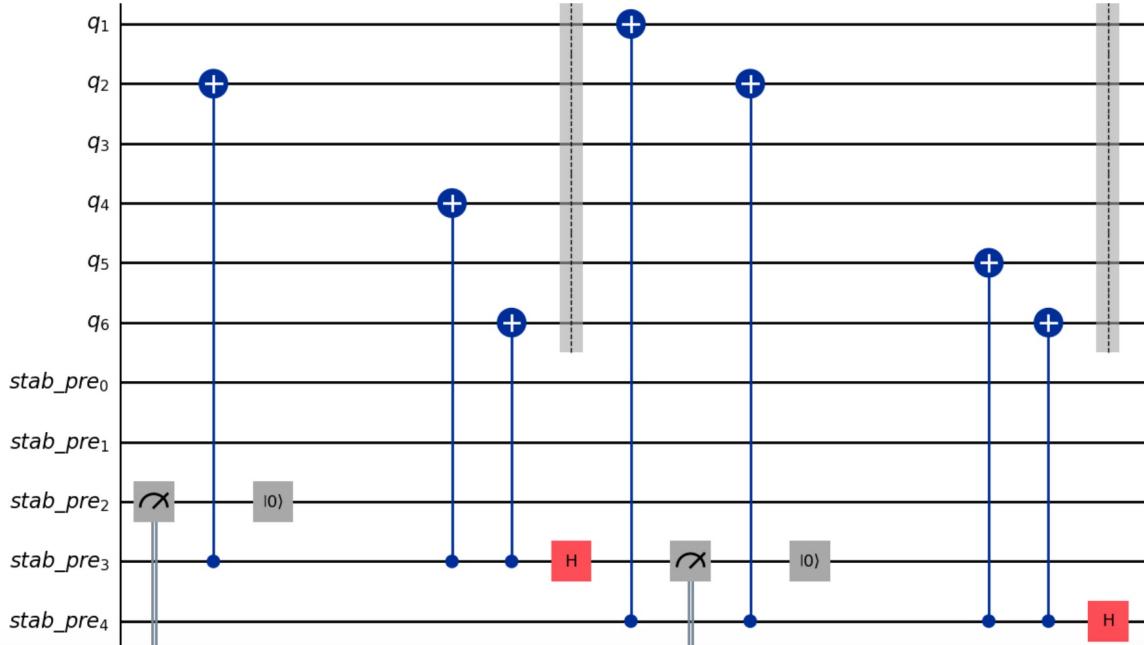


Figure 4: Part of the S_X^1 and the entirety of the S_X^2 ($stab_pre_3$ and $stab_pre_4$ respectively) stabilizer generators applied to the 7 physical qubits.

The measurement of each ancillary qubit is mapped onto a single-bit classical register. The classical registers are single-bit due to the requirements of the comparison function in Section 2.5.

The function used for drawing the circuit, `qc.draw("mpl")`, reorders and move gates for aesthetic and optimisation reasons, therefore it can be beneficial to view the code for stabilizer generator measurements.

#Apply Z stabilizers

```
qc.h(stab_pre[0])
qc.cz(stab_pre[0], q[0])
qc.cz(stab_pre[0], q[2])
qc.cz(stab_pre[0], q[4])
qc.cz(stab_pre[0], q[6])
qc.h(stab_pre[0])
qc.barrier(q)
qc.h(stab_pre[1])
qc.cz(stab_pre[1], q[1])
qc.cz(stab_pre[1], q[2])
qc.cz(stab_pre[1], q[5])
qc.cz(stab_pre[1], q[6])
qc.h(stab_pre[1])
## Error detectable ##
qc.barrier(q)
qc.h(stab_pre[2])
qc.cz(stab_pre[2], q[3])
qc.cz(stab_pre[2], q[4])
qc.cz(stab_pre[2], q[5])
qc.cz(stab_pre[2], q[6])
qc.h(stab_pre[2])
```

(a) The code for the S_Z measurements.

#Apply X stabilizers

```
qc.h(stab_pre[3])
qc.cx(stab_pre[3], q[0])
qc.cx(stab_pre[3], q[2])
qc.cx(stab_pre[3], q[4])
qc.cx(stab_pre[3], q[6])
qc.h(stab_pre[3])
qc.barrier(q)
qc.h(stab_pre[4])
qc.cx(stab_pre[4], q[1])
qc.cx(stab_pre[4], q[2])
qc.cx(stab_pre[4], q[5])
qc.cx(stab_pre[4], q[6])
qc.h(stab_pre[4])
qc.h(stab_pre[5])
qc.cx(stab_pre[5], q[3])
qc.cx(stab_pre[5], q[4])
qc.cx(stab_pre[5], q[5])
qc.cx(stab_pre[5], q[6])
qc.h(stab_pre[5])
```

(b) The code for the S_X measurements.

Figure 5: Coding for the initial stabilizer generator measurements. Note that the error can be implemented after the first set of stabilizer measurements, shown in (a).

Both the X and Z stabilizer measurements use ancillary qubits initialized in the $|0\rangle$ state. A Hadamard gate is then applied to prepare the ancillary qubit in the $|+\rangle$ state, as shown in Fig. 3 and Fig. 4.

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle. \quad (3)$$

For the S_Z stabilizer generator measurements, once the ancillary qubit is in the $|+\rangle$ state, control Z (cz) gates are applied; the physical qubits are the control and the ancillaries the target. If the relevant physical qubit is in the $|0\rangle$ state or has an even number of $|1\rangle$ states, then the cz gate introduces no overall change to the ancillary. The subsequent Hadamard gate is then applied, resolving the ancillary qubit in the $|0\rangle$ state and this is recorded as 0 in the classical register. However, if the physical qubit in the $|1\rangle$ state, or has an odd number of $|1\rangle$ states, then the cz gate results in a phase flip of the ancilla,

$$CNOT_{p,a} \left(|1\rangle, \frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) = \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right)_a = |-\rangle_a, \quad (4)$$

where p is the physical qubit and a is the ancillary qubit. The second Hadamard gate is then applied converting phase flips to bit flips,

$$H|-\rangle = |1\rangle. \quad (5)$$

The measured $|1\rangle$ state is associated with the -1 eigenvalue of the computational basis and thus recorded as a 1 in the classical register - the error is detected.

For the S_X stabilizer generator measurements, the ancillary qubit is again transformed into the $|+\rangle$ state. The control X (CNOT) gates are then applied with the ancillary qubit; the physical qubits are now the targets and the ancillaries the control. If the physical qubits is in the $|+\rangle$ state or there are an even number of $|-\rangle$ states then the CNOT gate introduces no overall change to the ancillary. However, if the physical qubit is in the $|-\rangle$ state, or has an odd number of $|-\rangle$ states, then the CNOT gate causes the ancilla to accumulate a phase flip,

$$CNOT_{a,p} \left(|+\rangle_a, |-\rangle_p \right) = \left(|-\rangle_a, |+\rangle_p \right). \quad (6)$$

where a is the ancillary qubit and p is the physical qubit. The second Hadamard gate is then applied converting phase flip to bit flip,

$$H|-\rangle = |1\rangle. \quad (7)$$

The measured $|1\rangle$ state is associated with the -1 eigenvalue of the computational basis and thus recorded as a 1 in the classical register - the error is detected.

The syndromes for each type of error were classified according to the paper from Lu and Marinescu [5]. The syndromes refer strictly to single qubit errors.

Error	Syndrome	Error	Syndrome	Error	Syndrome
X_1	000001	Z_1	001000	Y_1	001001
X_2	000010	Z_2	010000	Y_2	010010
X_3	000011	Z_3	011000	Y_3	011011
X_4	000100	Z_4	100000	Y_4	100100
X_5	000101	Z_5	101000	Y_5	101101
X_6	000110	Z_6	110000	Y_6	110110
X_7	000111	Z_7	111000	Y_7	111111

Table 1: The syndrome for each type of the three errors. Bit-flip errors, X , are detected by the S_Z generators. Phase-flip errors, Z , are detected by the S_X generators. Bit-and-phase flip errors are detected by both.

The Steane code uses six stabilizer generators to detect errors, with each generator completing parity checks on the seven physical qubits. Consider an arbitrary error on a qubit. The state of the qubit is unknown and will be in the state $|\psi\rangle$, as direct measurement on the qubit would collapse the qubit. The error, E , also unknown, acts on the qubit,

$$E|\psi\rangle. \quad (8)$$

The post-error stabilizer generator, S , is then applied,

$$SE|\psi\rangle = \pm 1ES|\psi\rangle. \quad (9)$$

The stabilizer generator will either commute or anticommute with the error because the error will be characterized by one of the Pauli group, due to the discretization of errors after measurement. If they commute the syndrome bit will give 0 and if they anticommute the syndrome bit will give 1; demonstrating the presence of an error as aforementioned. Now consider that the error was a bit-flip error occurring on the first qubit (q_0). The only stabilizer generator that entangles with the first qubit is S_Z^1 and following Eq. 8 the error and stabilizer generator will anti-commute:

$$XZ = -ZX. \quad (10)$$

This anticommutation results in a -1 eigenvalue which is recorded as a 1 in the classical register. The only other stabilizer generator that entangles with q_0 is S_X^1 , however, X will commute with X resulting in a $+1$ eigenvalue and a 0 in the classical register, not detecting the error. The S_Z generators will give 001 while the S_X generators will give 000, resulting in the syndrome 000001 as seen in table 1. The order of the S_Z^1 and S_X^1 generators is important when using table 1 however, the respective variations of the X and Z stabilizers do not matter as they commute.

When the stabilizer generators are first measured, the resulting syndromes reflect the modelled noise which is inherent in the stabilizer method of AerSimulation.

```
syn_pre: 000111 | syn_post: 011111 | error: 011000 | syn_check: 000111 (Count: 256)
syn_pre: 010111 | syn_post: 001111 | error: 011000 | syn_check: 010111 (Count: 233)
syn_pre: 000000 | syn_post: 011000 | error: 011000 | syn_check: 000000 (Count: 251)
syn_pre: 010000 | syn_post: 001000 | error: 011000 | syn_check: 010000 (Count: 260)
phase-flip error on the third qubit (q[2])
```

Table 2: An example of possible simulation results given a random error. Syn_pre represents the pre-error stabilizer generator measurements.

The four variations of the initial stabilizer generator measurement demonstrate the modelled noise, however, this initial discrepancy is inconsequential and the error implemented is duly corrected regardless of the simulated noise.

2.4 Error Implementation

Numpys random() function was used to generate the arbitrary error in this circuit.

```
### apply error ###
def apply_random_error(qc, q):
    """Applies a random single-qubit error (I, X, Y, or Z) to one random qubit."""
    error_gates = ['i', 'x', 'y', 'z']
    random_gate = np.random.choice(error_gates)
    random_qubit = np.random.randint(0, 7)

    if random_gate == 'y':
        qc.y(q[random_qubit])

    if random_gate == 'z':
        qc.z(q[random_qubit])

    elif random_gate == 'x':
        qc.x(q[random_qubit])
    #Identity has no effect
    apply_random_error(qc, q)
```

Figure 6: The code generating a random error on a single qubit.

The error is applied after the first stabilizer generator measurements, although, the error can be applied as earlier as shown in Fig. 4 and it would still be detectable. This small range in which the error can be applied and remain detectable is still very limited and does not reflect a realistic outlook of what a circuit requires in order to be considered fault tolerant.


```
for i in range(6):
    qc.measure(stab_pre[i], syn_pre_bits[i][0])
    qc.reset(stab_pre[i])
```

(a) The code for the classically storing the pre-error measurements.

```
for i in range(6):
    qc.measure(stab_post[i], syn_post_bits[i][0])
    qc.reset(stab_post[i])
```

(b) The code for the classically storing the post-error measurements.

Figure 7: The code for storing stabilizer generator measurements in single bit classical registers.

2.5 Error detection and correction

After the application of the error, the stabilizer generators were measured again and the syndromes were stored in single-bit classical registers.

The inherent noise generated by the AerSimulator means that one stabilizer measurement of the error is insufficient in determining the error introduced into the circuit. Instead, as mentioned in Section 2.3, the two measurements must be compared. The classical XOR gate could be replicated as a quantum gate by applying X gates to another ancillary qubit, initialised in the state $|0\rangle$, depending on the value of single-bit classical registers.

```
for i in range(6):
    with qc.if_test((syn_pre_bits[i], 1)):
        qc.x(e_ancilla[i])
    with qc.if_test((syn_post_bits[i], 1)):
        qc.x(e_ancilla[i])

qc.measure(e_ancilla, error)
qc.barrier(q)
```

Figure 8: The code simulating a quantum XOR gate. The QISkit function `if_test` is used to conditionally apply a quantum gate mid circuit [6].

An error has occurred if the measurement before the error is not equivalent to the measurement after the error. Therefore, a 1 is only recorded in the 'error' classical register if there is a difference between the two stabilizer classical registers. For example, if both syndromes had a 1 then the x gate would be applied twice flipping the original ancillary qubit into the $|1\rangle$ state and then back to the $|0\rangle$ state, thus resulting in a 0 value in the classical register. The comparison between the classical registers mid-circuit explains the necessity of single-bit classical registers in pre-and-post error stabilizer generator measurements.

The six-bit classical register, 'error', obtained from the comparison of the pre-and-post error measurements was used to then correct the error mid-circuit. Utilising Table 1, the syndromes are treated in binary in order to determine the type of error and which qubit was affected.

```

trivial = '000000'
value = int(trivial, 2)
with qc.if_test((error, value)):
    pass

X = np.arange(1, 8)
Z = np.arange(1, 8)

### Apply X corrections ###
for i in range(7):
    with qc.if_test((error, int(X[i]))):
        qc.x(q[i])

### Apply Z corrections ###
for i in range(7):
    syndrome_z = Z[i] * 8
    with qc.if_test((error, int(syndrome_z))):
        qc.z(q[i])

### Apply Y corrections ###
for i in range(7):
    syndrome_y = 9 * (i + 1)
    with qc.if_test((error, int(syndrome_y))):
        qc.y(q[i])

```

Figure 9: The code for applying corrections based on the binary syndromes.

The error, which is from the Pauli group after discretization, is corrected by applying the same Pauli.

$$XX = I. \quad (10)$$

After the correction is applied a final stabilizer generator measurement was taken in order to demonstrate the original state after the pre-error measurement was returned. The same stabilizer generator method shown in Fig 5 was used.


```

syn_pre: 000000 | syn_post: 100100 | error: 100100 | syn_check: 000000 (Count: 255)
syn_pre: 010111 | syn_post: 110011 | error: 100100 | syn_check: 010111 (Count: 253)
syn_pre: 000111 | syn_post: 100011 | error: 100100 | syn_check: 000111 (Count: 258)
syn_pre: 010000 | syn_post: 110100 | error: 100100 | syn_check: 010000 (Count: 234)
phase-flip and bit-flip error on sevent qubit (q[3])

```

Table 3: Another example of possible simulation results given a random error.

The aptly name 'syn_check' is the syndrome (stored in a classical register) that confirms the original state was returned.

3 Limitations

There are many limitations of the quantum circuit demonstrated. The large number of circuit components and the capability of only detecting and correcting one error on one qubit are inherent to this method of error detection using the Steane code. The chance of noise affecting the quantum circuit increases as the size of the circuit increases, therefore, there would likely be an increase in errors. The method of using these syndromes to classify error only works for single qubit errors because the syndrome corresponding to errors acting on multiple qubits may be equivalent to that of a single qubit error. Furthermore, the error location is known and if the error were to occur elsewhere it would not be detected or an incorrect gate would be applied as a correction attempt. Ultimately, this circuit serves an introductory example of quantum error correction with the Steane code, demonstrating some of the key concepts of the topic such as stabilizer formalism and syndromes as measurement outcomes.

4 Appendix

```

#!pip install qiskit
#!pip install qiskit-aer matplotlib pylatexenc
import numpy as np
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit_aer import AerSimulator

q = QuantumRegister(7, 'q') #Initialise 7 physical qubits.
stab_pre = QuantumRegister(6, 'stab_pre') # The 6 qubits forming the stabilizer generators.
stab_post = QuantumRegister(6, 'stab_post')
syn_pre_bits = [ClassicalRegister(1, f'pre_{i}') for i in range(6)] # Classical bits to store syndrome
syn_post_bits = [ClassicalRegister(1, f'post_{i}') for i in range(6)] # Classical bits to store syndrome
e_ancilla = QuantumRegister(6, 'e_ancilla') # Ancilla qubits for determining error.
error = ClassicalRegister(6, 'error')
stab_check = QuantumRegister(6, 'stab_check') # Confirmation measurements.
syn_check = ClassicalRegister(6, 'syn_check')

qc = QuantumCircuit(q, stab_pre, stab_post, e_ancilla,
                    *syn_pre_bits, *syn_post_bits, error, stab_check, syn_check)

### initialise data qubit ###
qc.h(q[0])

### Arbitrary information qubit encoded into 7 physical qubits - 1 logical qubit ###
qc.cx(q[0], q[5])
qc.cx(q[0], q[6])
qc.h(q[2])
qc.cx(q[2], q[0])
qc.cx(q[2], q[4])
qc.cx(q[2], q[6])

```

```

qc.h(q[1])
qc.cx(q[1], q[2])
qc.cx(q[1], q[5])
qc.h(q[3])
qc.cx(q[3], q[4])
qc.cx(q[3], q[5])
qc.cx(q[3], q[6])

qc.barrier(q)

### Apply Z stabilizers ###
qc.h(stab_pre[0])
qc.cz(stab_pre[0], q[0])
qc.cz(stab_pre[0], q[2])
qc.cz(stab_pre[0], q[4])
qc.cz(stab_pre[0], q[6])
qc.h(stab_pre[0])
qc.barrier(q)
qc.h(stab_pre[1])
qc.cz(stab_pre[1], q[1])
qc.cz(stab_pre[1], q[2])
qc.cz(stab_pre[1], q[5])
qc.cz(stab_pre[1], q[6])
qc.h(stab_pre[1])
## Error detectable ##
qc.barrier(q)
qc.h(stab_pre[2])
qc.cz(stab_pre[2], q[3])
qc.cz(stab_pre[2], q[4])
qc.cz(stab_pre[2], q[5])
qc.cz(stab_pre[2], q[6])
qc.h(stab_pre[2])
#Apply X stabilizers
qc.h(stab_pre[3])
qc.cx(stab_pre[3], q[0])
qc.cx(stab_pre[3], q[2])
qc.cx(stab_pre[3], q[4])
qc.cx(stab_pre[3], q[6])
qc.h(stab_pre[3])
qc.barrier(q)
qc.h(stab_pre[4])
qc.cx(stab_pre[4], q[1])
qc.cx(stab_pre[4], q[2])
qc.cx(stab_pre[4], q[5])
qc.cx(stab_pre[4], q[6])
qc.h(stab_pre[4])
qc.h(stab_pre[5])
qc.cx(stab_pre[5], q[3])
qc.cx(stab_pre[5], q[4])
qc.cx(stab_pre[5], q[5])
qc.cx(stab_pre[5], q[6])
qc.h(stab_pre[5])

for i in range(6):
    qc.measure(stab_pre[i], syn_pre_bits[i][0])

### apply error ###
def apply_random_error(qc, q):
    """Applies a random single-qubit error (I, X, Y, or Z) to one random qubit."""

```

```

error_gates = ['i', 'x', 'y', 'z']
random_gate = np.random.choice(error_gates)
random_qubit = np.random.randint(0, 7)

if random_gate == 'y':
    qc.y(q[random_qubit])

if random_gate == 'z':
    qc.z(q[random_qubit])

elif random_gate == 'x':
    qc.x(q[random_qubit])
#Identity has no effect
apply_random_error(qc, q)

### Apply Z stabilizers again to detect bit-flip error ###
qc.h(stab_post[0])
qc.cz(stab_post[0], q[0])
qc.cz(stab_post[0], q[2])
qc.cz(stab_post[0], q[4])
qc.cz(stab_post[0], q[6])
qc.h(stab_post[0])
qc.barrier(q)
qc.h(stab_post[1])
qc.cz(stab_post[1], q[1])
qc.cz(stab_post[1], q[2])
qc.cz(stab_post[1], q[5])
qc.cz(stab_post[1], q[6])
qc.h(stab_post[1])
qc.barrier(q)
qc.h(stab_post[2])
qc.cz(stab_post[2], q[3])
qc.cz(stab_post[2], q[4])
qc.cz(stab_post[2], q[5])
qc.cz(stab_post[2], q[6])
qc.h(stab_post[2])
#Apply X stabilizers again to detect phase-flip error.
qc.h(stab_post[3])
qc.cx(stab_post[3], q[0])
qc.cx(stab_post[3], q[2])
qc.cx(stab_post[3], q[4])
qc.cx(stab_post[3], q[6])
qc.h(stab_post[3])
qc.barrier(q)
qc.h(stab_post[4])
qc.cx(stab_post[4], q[1])
qc.cx(stab_post[4], q[2])
qc.cx(stab_post[4], q[5])
qc.cx(stab_post[4], q[6])
qc.h(stab_post[4])
qc.h(stab_post[5])
qc.cx(stab_post[5], q[3])
qc.cx(stab_post[5], q[4])
qc.cx(stab_post[5], q[5])
qc.cx(stab_post[5], q[6])
qc.h(stab_post[5])

for i in range(6):
    qc.measure(stab_post[i], syn_post_bits[i][0])

```

```

qc.barrier(q)

### Effective XOR onto error ancilla ###

for i in range(6):
    with qc.if_test((syn_pre_bits[i], 1)):
        qc.x(e_ancilla[i])
    with qc.if_test((syn_post_bits[i], 1)):
        qc.x(e_ancilla[i])

qc.measure(e_ancilla, error)
qc.barrier(q)

### Apply gates (as corrections) based on syndrome ###

trivial = '000000'
value = int(trivial, 2)
with qc.if_test((error, value)):
    pass

X = np.arange(1, 8)
Z = np.arange(1, 8)

### Apply X corrections ###
for i in range(7):
    with qc.if_test((error, int(X[i]))):
        qc.x(q[i])

### Apply Z corrections ###
for i in range(7):
    syndrome_z = Z[i] * 8
    with qc.if_test((error, int(syndrome_z))):
        qc.z(q[i])

### Apply Y corrections ###
for i in range(7):
    syndrome_y = 9 * (i + 1)
    with qc.if_test((error, int(syndrome_y))):
        qc.y(q[i])

### Apply Z stabilizers again for confirmation ###
qc.h(stab_check[0])
qc.cz(stab_check[0], q[0])
qc.cz(stab_check[0], q[2])
qc.cz(stab_check[0], q[4])
qc.cz(stab_check[0], q[6])
qc.h(stab_check[0])
qc.measure(stab_check[0], syn_check[0])
qc.barrier(q)
qc.h(stab_check[1])
qc.cz(stab_check[1], q[1])
qc.cz(stab_check[1], q[2])
qc.cz(stab_check[1], q[5])
qc.cz(stab_check[1], q[6])
qc.h(stab_check[1])
qc.barrier(q)
qc.measure(stab_check[1], syn_check[1])
qc.h(stab_check[2])
qc.cz(stab_check[2], q[3])
qc.cz(stab_check[2], q[4])

```

```

qc.cz(stab_check[2], q[5])
qc.cz(stab_check[2], q[6])
qc.h(stab_check[2])
qc.measure(stab_check[2], syn_check[2])
#Apply X stabilizers again for confirmation.
qc.h(stab_check[3])
qc.cx(stab_check[3], q[0])
qc.cx(stab_check[3], q[2])
qc.cx(stab_check[3], q[4])
qc.cx(stab_check[3], q[6])
qc.h(stab_check[3])
qc.measure(stab_check[3], syn_check[3])
qc.barrier(q)
qc.h(stab_check[4])
qc.cx(stab_check[4], q[1])
qc.cx(stab_check[4], q[2])
qc.cx(stab_check[4], q[5])
qc.cx(stab_check[4], q[6])
qc.h(stab_check[4])
qc.measure(stab_check[4], syn_check[4])
qc.barrier(q)
qc.h(stab_check[5])
qc.cx(stab_check[5], q[3])
qc.cx(stab_check[5], q[4])
qc.cx(stab_check[5], q[5])
qc.cx(stab_check[5], q[6])
qc.h(stab_check[5])
qc.measure(stab_check[5], syn_check[5])

### Simulation ###

simulator = AerSimulator(method = "stabilizer")
compiled_circuit = transpile(qc, simulator)
job = simulator.run(compiled_circuit, shots=1000, memory = True)
result = job.result()
mem = result.get_memory()
counts = result.get_counts(qc)

for bitstring, count in counts.items():
    parts = bitstring.split()
    if len(parts) == 14: # 12 single-bit clregisters + 2 6 bit clregisters.
        syn_check_str = parts[0]
        error_str = parts[1]
        syn_post_bits_str = "".join(parts[2:8]) # syn_post bits
        syn_pre_bits_str = "".join(parts[8:14]) # syn_pre bits
        print(f"syn_pre: {syn_pre_bits_str} | "
              f"syn_post: {syn_post_bits_str} | "
              f"error: {error_str} | "
              f"syn_check: {syn_check_str} (Count: {count})")

if error_str == '000000':
    print("No error detected")

elif error_str == '000001':
    print('bit-flip error on the first qubit (q[0])')
elif error_str == '000010':
    print('bit-flip error on the second qubit (q[1])')
elif error_str == '000011':
    print('bit-flip error on the third qubit (q[2])')

```

```

elif error_str == '000100' :
    print('bit-flip error on the fourth qubit (q[3])')
elif error_str == '000101' :
    print('bit-flip error on the fifth qubit (q[4])')
elif error_str == '000110' :
    print('bit-flip error on the sixth qubit (q[5])')
elif error_str == '000111' :
    print('bit-flip error on the seventh qubit (q[6])')

elif error_str == '001000' :
    print('phase-flip error on the first qubit (q[0])')
elif error_str == '010000' :
    print('phase-flip error on the second qubit (q[1])')
elif error_str == '011000' :
    print('phase-flip error on the third qubit (q[2])')
elif error_str == '100000' :
    print('phase-flip error on the fourth qubit (q[3])')
elif error_str == '101000' :
    print('phase-flip error on the fifth qubit (q[4])')
elif error_str == '110000' :
    print('phase-flip error on the sixth qubit (q[5])')
elif error_str == '111000' :
    print('phase-flip error on the seventh qubit (q[6])')

elif error_str == '001001':
    print("phase-flip and bit-flip error on first qubit (q[0])")
elif error_str == '010010':
    print("phase-flip and bit-flip error on second qubit (q[1])")
elif error_str == '011011':
    print("phase-flip and bit-flip error on third qubit (q[2])")
elif error_str == '100100':
    print("phase-flip and bit-flip error on fourth qubit (q[3])")
elif error_str == '101101':
    print("phase-flip and bit-flip error on fifth qubit (q[4])")
elif error_str == '110110':
    print("phase-flip and bit-flip error on sixth qubit (q[5])")
elif error_str == '111111':
    print("phase-flip and bit-flip error on seventh qubit (q[6])")

qc.draw("mpl")

```


References

- [1] Steane, A.M. (1996) 'Simple quantum error-correcting codes,' *Physical Review A*, 54(6), pp. 4741–4751. <https://doi.org/10.1103/physreva.54.4741>.
- [2] AerSimulator - Qiskit AER 0.16.1. https://qiskit.github.io/qiskit-aer/stubs/qiskit_aer.AerSimulator.html.
- [3] Quantum Error Correction — Applications of Quantum Mechanics (no date). https://quantum.lassp.cornell.edu/lecture/quantum_error_correction.
- [4] Quan, D. et al. (2022) 'Implementation of Fault-Tolerant encoding circuit based on stabilizer implementation and “Flag” bits in steane code,' *Entropy*, 24(8), p. 1107. <https://doi.org/10.3390/e24081107>.
- [5] Lu, F. and Marinescu, D.C. (2007) 'Quantum error correction of time-correlated errors,' *Quantum Information Processing*, 6(4), pp. 273–293. <https://doi.org/10.1007/s11128-007-0058-1>.
- [6] IBM. (2025). QuantumCircuit class. [Online]. IBM Quantum Documentation. Available at: <https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.QuantumCircuit#quantumcircuit-class> [Accessed 22 April 2025].